- 本题学习大部分由**ChatGPT 辅助完成**，另外还观看了观看**B 站、Youtube相关视频**。
- ChatGPT对话链接：https://chatgpt.com/share/6acf1ba6-1610-83ee-99ce-0d0ec6d767e1
- B站视频：李沐有关如何读论文、Alex 论文的视频。
- YouTube视频：3Blue1Brown的Deep Learning系列全视频。

花卉小能手

第二部分

- 一张RGB三通道彩色图片是如何被处理成可用于训练的数据？张量又是什么？
  - **处理方式**：
    - 一张普通 RGB 彩色图片中，每个像素使用 **三个数值** 表示颜色：(R, G, B)
    - **每个颜色分量用 8 个二进制位存储**，每个分量通常可以取整数 0 ~ 255。
      - 数值越大，颜色越明显，分量为 0 表示黑色。
    - **颜色通道**：图像某一种颜色分量所对应的数值信息。
      - 一张宽度为 W、高度为 H 的 RGB 图片，每个颜色通道都可以表示为一个 $H \times W$ 的二维数值矩阵。
    - PyTorch中一张 RGB 彩色图片可以表示为形状为 $(3, H, W)$ 的张量。
      - 例：`array[0][H - 1][W - 1] = 123`就表示 R 通道在第 H 行，第 W 个像素的数值为 123。
  - **张量**：在机器学习中，张量是 **一种表示多维数值数组的数据结构**。
    - 本质是多维数组，便于索引多维数据。
    - **维度数量**：一个标量，表示需要几个下标定位元素。
      - **轴**：张量的特定维度。
        - **轴的长度**：在某个特定维度定位所有元素需要的整数个数。
    - **形状**：一个数组，衡量每个维度的长度。
    - **元素数量**：表示张量有多少个元素。
      - 数值上等于各轴的长度的积。
- 一个卷积神经网络分为哪几层？
  - 通常分为**卷积层、激活层、池化层、展平层、全连接层**。
    - CNN 没有固定的层数，也不是必须包含上面说的的所有层。
- 什么是卷积？数学严格意义上的卷积和神经网络中的卷积操作有区别吗？卷积核、步长是什么？
  - **卷积核**：一个较小的、CNN用于对照片的局部区域进行加权运算的数值数组
  - **步长**：计算窗口在图片上移动时，每次移动的像素个数。
  - **卷积**：以卷积核为计算窗口，在输入图像上滑动并进行加权计算的操作。
  - 数学上进行卷积运算要先将**卷积核的各个维度翻转**（对于二维离散卷积，相当于先把卷积核上下、左右各翻转一次），再进行局部加权运算，而神经网络中卷积操作**不需要翻转卷积核**。
- 池化有哪些类型，其作用分别有什么？感受野是什么？
  - 池化的 **类型** 及其 **作用**：
    - **最大池化（Max Pooling）**：对于每个局部区域，取其中的最大值作为输出。
      - 作用：**减小计算量、保留区域内最强的特征响应、减少细节和噪音等对模型的影响，抑制过拟合等**。
    - **平均池化（Average Pooling）**：对局部区域内的所有数值求算术平均值，将平均值作为输出。

        - 作用：**汇总整体特征信息、使区域的特征更“平滑”、把某个区域的特征浓缩到一个值，可以防止过拟合等等**。
    - **自适应池化（Adaptive Pooling）**人为指定池化输出的尺寸大小，模型根据输入和输出目标尺寸，自动确定如何划分池化区域。
        - 作用：**便于控制输出尺寸**。
        - 实践中常结合最大池化和平均池化使用。
    - **全局平均池化（Global Average Pooling）**：平均池化的一种，对每个输入通道的整个空间区域取平均值。
        - 作用：**大幅减少后续计算量、汇总整张特征图、降低过拟合风险等**
    - **全局最大池化（Global Max Pooling）**：最大池化的一种，对每个输入通道的整个空间区域取最大值。
        - 作用：**大幅减少后续计算量、保留整张特征图的最强特征、降低过拟合风险等**
  - **感受野**：：某一层特征图上某个位置的感受野，是输入图像中能够影响该位置输出数值的空间区域，也就是得出这个位置时，计算窗口覆盖的区域。
- 激活层在此的作用？最后卷积神经网络会输出什么？
  - **激活层的作用**：通过使用激活函数，为卷积神经网络引入非线性关系，从而提升网络的复杂度和模型的识别能力。
  - **输出：**一个批次各张图片各个类别的"得分"。

第三部分

- **查看数据**：check_data.py
- **隔离异常数据**：quarantine_abnormality.py
- **划分训练集和验证集**：split_data.py
- **调整图片大小、在线数据增强**：data_processing_and_augmentation.py

第四部分

- 什么是超参数？如何设定？
  - **超参数**：人为设置的模型的参数，训练过程中模型无法通过学习得到、改变。
  - **设定**：
    - **先根据数据及大小等设定合适的初始值**，再结合训练集、验证集的损失、准确率等 **随着训练人工调控，逐步优化。**
    - 一些超参数，如训练轮次，可以通过早停机制等自动调控。
    - **根据要求设定**，如本题给定卷积层层数和卷积核大小。
- 什么是优化器，有哪些类型？
  - **优化器**：按照特定算法更新神经网络模型参数，从而尝试降低损失的工具。
  - **类型**：
    - **小批量随机梯度下降（Mini-batch SGD）**：计算一小批训练数据的平均梯度，然后进行梯度下降。
      - SGD 是随机梯度下降，意为随机抽取训练样本估计梯度，但由于一次使用一个样本，更新频繁，且波动较大
      - **在Pytorch 中，称小批量梯度下降为 SGD**。
    - **动量法（Momentum）**：利用历史梯度信息，使参数持续向某方向更新时更新加快，在参数收敛时加快收敛。
    - **结合动量与自适应学习率(Adam)(Adaptive Moment Estimation)**：利用过去的梯度方向和梯度平方大小，调整梯度的大小和有效学习率（实际上可以认为只调整了梯度）从而 **自动**

调整步长，加快参数更新、收敛。

- 小登在上文列出了常见的优化器类型与主要作用。之前学习、做题过程中，小登记录过其他优化器与大致原理，书写过有关动量法和 Adam 的理解。将此内容放在下面，由于 **内容较长，学长可视情况浏览、批改，如能指出小登理解不当之处，小登万分感激**。

  - **自适应梯度算法（AdaGrad）（Adaptive Gradient）**：根据训练过程中观察到的梯度，自动调整不同参数的有效学习率。

$$G_t = \sum_{k=1}^{t} g_k^2$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{G_t} + \varepsilon} g_t$$

    - G_t：截至当前的历史梯度平方和。
    - $varepsilon$：防止分母为零的一个很小的正数。
    - g_t：当前梯度。
    - **主要局限： 梯度平方和会不断累积，可能使有效学习率越来越小，最终更新过慢**。

  - **改进自适应学习率(RMSprop)(Root Mean Square Propagation)**：

    - s_t：当前的梯度平方统计量。
    - ρ：控制历史信息保留程度的系数，通常在 0～1 之间。
    - 主要作用： 根据近期梯度大小自动调整不同参数的更新幅度，减轻 AdaGrad 有效学习率持续减小的问题。

  - **动量法：**

    - **普通梯度下降的不足：** 由于每次参数更新只看当前的梯度， *步长比较固定，长期向一个方向更新步长时，更新速度可能较慢，改变方向时* 损失震荡等的可能性较大。

    - **思考：**

      - 因此，我们希望调整参数更新的公式，在公式中引入新的变量，使它体现先前的梯度对当前更新的影响，从而防止参数更新方向与步长的剧变，也能加快长期向一个方向更新的速度。
      - 不妨把当前的梯度记作 g_t。普通梯度下降中，我们直接使用 $\eta * g_t$ 更新参数,结合上一条思考，不妨把 g_t 改为即能体现当前梯度，又能体现上一轮梯度的变量 $\beta v_{t-1} + g_t$,我们用 v_t 表示这个变量，它即受 v_{t-1} 也受 g_t 的影响，这就是动量法。

    - **动量法：** 在普通梯度下降的基础上，引入

$$w_{t+1} = w_t + \eta v_t$$

$$v_t = \beta v_{t-1} + g_t$$

      - 其中 w_{t+1} 表示下一轮的参数；
      - w_t 是当前的参数；
      - η 是学习率；
      - v_t 是当前的动量；

    - v_{t-1} 是上一轮的动量；
    - β 是动量系数（0 ~ 1），表示上次更新对当前的影响；
    - g_t 是当前梯度。
  - 根据公式，我们得知动量法保留了先前运动的状态。
  - 当前梯度方向与之前相同时，即 g_t 与 g_{t-1} 、 g_{t-2} 等同号,那么 v_t 极有可能增大，从而增大参数更新的步长，加快参数更新。
  - 当梯度方向改变时， v_t 可能并不会变号，即使变号，绝对值也不会太大，从而减少损失震荡，有利于收敛。
- **Adam 优化器**：
  - 除参数震荡、梯度频繁改变方向外，可能还有 **不同参数的梯度数值差异较大、某些梯度长期过大或过小等问题** 。
  - **思考：**
    - 既然梯度的数值会突变，某些参数长期过大/过小，我们能否引入一个变量，让它体现先前梯度更新的幅度，同时适当减小/增长变量更新的步长？
    - 梯度有正有负，我们可以 **取绝对值或者平方后开根** ，来衡量梯度的大小，但是由于 **绝对值不利于求导** ，于是我们选择平方。引入变量 v_t ,使得 $v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$ ，其中为了 **不让 v_t 过大** ，我们选择让 v_{t-1} 、 g_t^2 **的权重之和为1** 。
    - 那么把这个变量放在哪个位置，可以在梯度小/大时，增加/减小步长呢？
    - 不妨把它 **放在更新项的分母** 上，为了不让 **平方导致数量级剧变** ，我们给它 **开根号**，为了 **不让分母为0** ，我们还要在 **分母加上极小量** ϵ 。
    - 问题又来了，第一代的 **历史值为零** ，我们，我们还 **给当前的梯度乘了小于1的** $beta_2$，那么这次 **更新幅度可能太大** 了！我们不妨引入 $hat v_t = \frac{v_t}{1-\beta_2^t}$,从而避免 **初始偏差**。后期数值稳定时，为了减弱 β_2 的影响，我们取它的 **t 次方** 。
    - 结合对 v_{t-1} 、 g_t^2 的权重之和为1的处理，你是否还记得动量法还有一个 v_{t-1} ，在 Adam 优化器，我们叫它 m_t ,同样为了 **不让 m_t 过大**,我们也引入 m_{t-1} 、 g_t 权重之和为 1 。
    - 同样因为第一次没有历史值，直接乘以小于一的 β_1 使得 m_t 较小，我们也来个 $\hat{m}_t = \frac{m_t}{1-\beta_1^t}$,对它进行修正。
    - 于是，我们得到了 **Adam优化器**

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

    - m 相当于 动量法中的 v；
    - v_t 用于调整步长；
    - ϵ 极小，用于防止 v_t = 0；
    - $\hat{m}$ 、 $\hat{v}_t$ 是为了减弱第一代历史值为零带来的影响（初始偏差），随训练时间增长 β_1^t 、 β_2^t 会趋近于零。
    - 当前 mini-batch ——→ 计算梯度 g ——→ 看看最近梯度总体朝哪个方向（ m ） ——→ 看看最近梯度通常有多大（ v ） ——→ 修正刚开始训练时的偏差 ——→ 根据梯度大小自动调整实际步长 ——→ 更新参数
- DataLoader 与 DataSet是什么及其使用？（可调库）
    - **Dataset（数据集对象）**：一个 **按照编号读取样本特征、标签），并对样本预处理（放缩尺寸、数据增强等）** 的对象。
    - **DataLoader（数据加载器）**：一个从 **Dataset 中 取得样本，将它们按照 Batch 大小组织成批次，并且可以控制样本读取顺序** 的类。

第五部分

- 1.相信大家有疑问，我如果一直堆叠网络的层数，岂不是能无限的提高模型的能力？模型层数越多、结构越深，性能就一定越好吗？请自行查阅资料学习。
    - 模型的能力分为表达能力、优化能力和泛化能力。堆叠层数主要提升模型的表达能力。
    - **过拟合问题**：模型表达能力过强，训练过程中对训练集的细节和噪音表达过强，但是面对新数据，已有的参数不能做出较好的预测，此时模型的训练集损失会更低、准确率更高，但验证集、测试集的准确率可能不变，甚至下降，损失可能不变，甚至增加。
    - **优化问题**：网络过深，复合函数复合层数过多，由复合函数的链式求导法则，当部分导数过大时，可能产生**梯度爆炸**，部分导数过小时，可能产生**梯度消失**。
    - **计算资源和储存要求随网络加深增大**。网络过深时，可能**对训练设备会有更高的要求**，也可能**会需要更多的训练时间**。
- 3. 模型训练好了该如何使用呢？难道每次使用前都需要重新训练一次？请学习模型的保存与载入方法，将自己的模型保存在本地，自己在网上找到图片后丢给模型分类。
    - 不需要重新训练，可以通过`torch.save()`和`.load_state_dict`保存、调用模型状态。
    - 对于 **dadelion, rose, tulip** 分类效果好，但**可能由于某些 daisy 和 sunflower 有些相似，以及拍摄角度、背景的问题** 后两者输出概率的差别相对较小。